ML_Honey_trap: An Adaptive Machine Learning Framework for Intelligent Honeypot Threat Classification and Engagement

First Author*, Second Author*
*Department of Computer Science
University Name
City, Country
{first.author, second.author}@university.edu

***Abstract*—Honeypots face a design paradox: high-interaction systems elicit richer attacker behavior but carry greater compromise risk, whereas low-interaction systems are safer but easier for skilled attackers to recognize [1], [12]. Prior work tends to emphasize either attacker classification or adaptive response, and rarely evaluates the two together [1]–[3], [7]. We present ML_Honey_trap, a four-layer framework that couples a Cowrie SSH/Telnet honeypot with an XGBoost classifier that assigns each session to one of three threat levels (Scanner, Amateur, Advanced) and a response engine that adjusts engagement according to the predicted level. Because the public Shell Honeypot dataset is annotated with a severity index rather than threat classes, we define an explicit threshold-based mapping from severity to class and treat it as a heuristic, not ground truth. We describe a 15-feature behavioral representation, the training procedure, and an evaluation protocol with three hypotheses covering classification accuracy, engagement duration, and threat-intelligence quality, together with baselines and ablations. This draft reports no experimental results; all measurements are marked as pending.**

***Index Terms*—Honeypot, Machine Learning, XGBoost, Threat Classification, Cyber Deception, Intrusion Detection, Adaptive Security, Cowrie**

I. Introduction

Defending networks against increasingly capable adversaries requires more than perimeter controls: defenders also need to observe what attackers actually do once they reach a system. Honeypots, decoy systems deployed to attract and record malicious activity, serve this purpose. They have evolved from simple listeners that log connection attempts into emulated services that capture credentials, command sequences, and malware payloads, and more recently into systems that adapt their behavior to the attacker [1], [12].

A central difficulty is the honeypot design paradox. High-interaction honeypots expose a real operating environment and therefore yield realistic engagement, but they can be compromised and used as a launch point. Low-interaction honeypots emulate services in a restricted way, which is safer but makes them easier for a sophisticated attacker to fingerprint and abandon [1], [12]. The paradox is sharpened by evidence that attackers may themselves use AI to detect honeypots [8], so a static emulation is a weakening position over time.

Machine learning offers a way to ease the paradox. If a honeypot can estimate, early in a session, how skilled and how dangerous the visitor is, it can spend its engagement budget accordingly: minimal interaction for automated scanners, richer interaction for skilled operators. Work in this direction includes XGBoost-based attacker categorization over Cowrie logs [1], focal-loss-optimized XGBoost combined with flow features and automated firewall response [2], and reinforcement-learning approaches to adaptive deception [3], [7], [8]. Several of these studies also note a gap between laboratory accuracy and production behavior [1], and the community faces a shortage of labeled honeypot data [12].

Existing studies lean toward classification or toward adaptation. This paper proposes ML_Honey_trap, which connects the two: a classifier assigns a threat level from behavioral features of a live session, and the predicted level drives a rule-based response engine. The contributions of this draft are:

1) **Architecture.** A modular four-layer design (Deception, Analytics, Adaptation, Intelligence) with a defined data flow between layers.
2) **Label derivation.** An explicit, reproducible heuristic that maps the severity index of the Shell Honeypot dataset [10] to three threat classes, with its limitations stated.
3) **Evaluation protocol.** Hypotheses, metrics, baselines, and ablations to be executed, with every outcome left as a placeholder until measured.

Section II reviews related work, Section III describes the architecture, Section IV the implementation, Section V the evaluation protocol, Section VI discusses design implications and limitations, and Section VII concludes.

II. Related Work

A. Honeypot Evolution and Taxonomy

Surveys and systematizations of knowledge describe a progression from static, low-interaction honeypots toward high-interaction and adaptive designs, and more recently toward LLM-assisted systems [1], [12]. Within SSH and Telnet emulation, Cowrie [16] is widely used as the deception component

and is the base layer in several of the systems discussed below [1], [2], [5]. It records sessions as structured JSON events, which makes it suitable for feature extraction.

B. Machine Learning for Threat Classification

Reference [1] reviews supervised, unsupervised, and hybrid approaches to honeypot analytics and presents a Cowrie plus XGBoost system that assigns attackers to three categories: Scanner, Amateur, and Advanced. Reference [4] compares Random Forest (RF), Extra Trees, SVM, AdaBoost, Logistic Regression, and Naive Bayes on NSL-KDD [14] and finds RF to perform best, reporting 17,354 true positives, 235 false negatives, and 196 false positives. Reference [5] applies RF to 5,740 Cowrie log entries and pairs it with a Flask dashboard for real-time alerts. Reference [2] combines Cowrie, CICFlowMeter flow features, and a focal-loss-optimized XGBoost for attacker profiling, with automated response through an IPFire firewall. Gradient-boosted trees [13] are attractive for this setting because honeypot features are tabular, mixed-scale, and often class-imbalanced.

C. Reinforcement Learning for Adaptive Honeypots

Reinforcement learning (RL) treats engagement as a sequential decision problem. Reference [3] reports an RL-driven adaptive deception framework with 97% detection accuracy and a 3.8% false-positive rate, and reports that it outperforms Cowrie and T-Pot. Reference [7] uses Deep Q-Networks to balance data collection against the risk of deep compromise and evaluates a real deployment against Cowrie and HiHP. Reference [8] applies RL to response generation in Cowrie and addresses the problem of AI-based honeypot detection.

D. LLM-Enhanced and Multi-Service Deception

Reference [9] combines RL with LLM-generated deception and reports a 23% increase in engagement reward and a 55 percentage-point gain in intent-analysis accuracy relative to Wazuh. Reference [6] describes a multi-service deception engine (HTTP, FTP, SSH, SQL) with a bot-versus-human classifier, reporting 100% credential capture and visualization latency under 120 ms.

E. Datasets and Benchmarks

The Shell Honeypot Attack Request–Response dataset [10] contains 12,923 commands from 2021–22 and 4,236 from 2024, annotated with MITRE ATT&CK [15] techniques and a severity index $V_i \in [0, 4]$. MURHCAD [11] contains 132,425 attack events from 2,438 unique IP addresses across 95 countries, collected with Cowrie, Dionaea, and SentryPeer. The scarcity of labeled honeypot data is described as a "data desert" in [12].

F. Research Gap

Based on the works above, we identify three gaps that ML_Honey_trap targets: (i) classification and adaptation are usually studied separately; (ii) validation under production conditions is limited, as acknowledged in [1]; and (iii) labeled behavioral data with transparent labeling rules is scarce [12]. This draft addresses (i) architecturally, and (iii) by stating a labeling rule explicitly; (ii) is left to the evaluation protocol and future work.

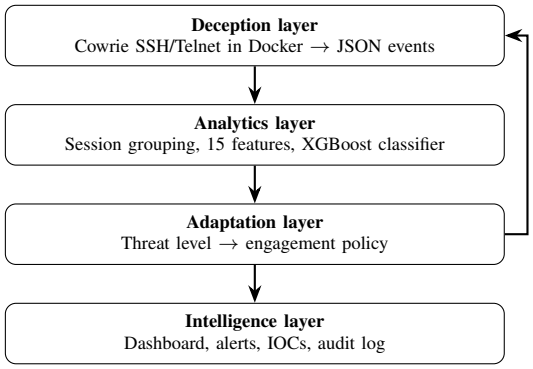


Fig. 1. ML_Honey_trap four-layer architecture. The feedback arrow carries the engagement policy back to the honeypot.

III. System Architecture

A. Overview

ML_Honey_trap is organized into four layers (Fig. 1). The *Deception layer* runs Cowrie and emits JSON events. The *Analytics layer* groups events into sessions, computes features (Section III-C), and performs classification (Section III-D). The *Adaptation layer* maps the predicted threat level to an engagement policy and feeds it back to the honeypot. The *Intelligence layer* exposes alerts, statistics, and indicators of compromise (IOCs) to analysts. Data flows from Cowrie logs to feature vectors to a threat level, which both selects the response and is stored for the dashboard.

B. Deception Layer: Cowrie

Cowrie is chosen because it emulates SSH and Telnet, records sessions as JSON, and can run in an isolated Docker container; it is the deception base in [1], [2], [5]. The honeypot is deployed with no route to production networks, and outbound traffic is blocked by default (see the Kill Switch in Section III-E). Each connection yields a session identifier, and events such as login attempts, command input, file downloads, file uploads, and session close are logged against it.

C. Analytics Layer: Feature Extraction

Each session is represented by 15 features: ten at session level (Table I) and five at payload level (Table II). Numeric features are standardized using statistics fitted on the training split only. Sessions with no commands (e.g., authentication-only probes) receive zero for count-based features and a sentinel value for timing features, which gradient-boosted trees handle natively. Feature choices follow the behavioral signals discussed in [1], [4]; their individual contribution is to be tested in the ablation study (Section V-F).

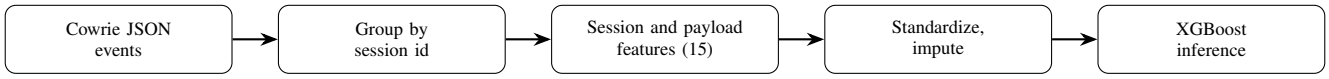


Fig. 2. Feature extraction pipeline from raw Cowrie events to a threat-level prediction.

TABLE I
SESSION-LEVEL FEATURES

Feature	Definition
`command_count`	Number of commands entered
`command_diversity`	Unique commands divided by total commands
`inter_command_delay_avg`	Mean time between consecutive commands
`session_duration`	Time from connect to close
`download_attempts`	Count of file-download events or download commands
`privilege_escalation`	Count of commands matching a privilege-escalation pattern list
`lateral_movement`	Count of commands that target other hosts (pattern list)
`file_upload_count`	Count of file-upload events
`unique_commands`	Absolute count of distinct commands
`failed_auth_attempts`	Count of failed login events

TABLE II
PAYLOAD-LEVEL FEATURES

Feature	Definition
`command_length_avg`	Mean character length of commands
`entropy_score`	Mean Shannon entropy of command strings
`base64_presence`	Count of commands containing base64-like blobs
`known_malicious_signatures`	Count of commands matching a signature list
`threat_db_matches`	Count of URLs/IPs/hashes found in a threat-intelligence source

D. Classification Layer: XGBoost

XGBoost [13] is selected for three reasons: it handles tabular features with mixed scales, supports class weighting and custom objectives for imbalance, and offers fast inference and feature-importance output. Cowrie-based systems in [1], [2] use the same model family. Whether XGBoost outperforms RF [4], [5] on this task is an empirical question addressed in Section V.

Label derivation (heuristic). The Shell Honeypot dataset [10] annotates commands with a severity index $V_i \in [0, 4]$ and does not provide the three threat classes used here. We therefore *derive* labels with the rule in Table III. Since V_i is defined per command, a session-level label requires aggregation; we use the maximum V_i over a session's commands, so a session is labeled by its most severe action. [VERIFY: confirm that the dataset exposes session identifiers or an ordering from which sessions can be reconstructed; if not, label per command window instead and revise Table I accordingly]. The thresholds are design choices, not empirical findings, and a sensitivity analysis over alternative thresholds is part of the protocol.

Class imbalance. Scanner sessions are expected to dominate real honeypot traffic. Following [2], we will evaluate a focal-loss objective, which down-weights easy examples by a factor $(1 - p_t)^\gamma$:

$$\mathcal{L}_{\text{focal}} = -\alpha_t \, (1 - p_t)^\gamma \, \log(p_t), \tag{1}$$

where p_t is the predicted probability of the true class. The focusing parameter γ and class weights α_t are tuned on validation data; plain class-weighted cross-entropy serves as the comparison.

TABLE III
LABEL DERIVATION RULE (HEURISTIC)

Severity range	Class	Interpretation
$V_i < 1.5$	Scanner	Automated, low sophistication
$1.5 \le V_i < 3.0$	Amateur	Semi-skilled, script-based
$V_i \ge 3.0$	Advanced	Skilled, multi-stage activity

E. Adaptation Layer: Response Engine

The response engine maps the predicted level to an engagement policy. The values below are *initial design choices* to be tuned, and the effects attributed to them are hypotheses (Section V-B), not findings.

- **Scanner:** log only, minimal interaction, short timeout (initial value: 5 min). *Hypothesis:* saves resources without reducing useful intelligence.
- **Amateur:** standard engagement, capture all commands and downloads, session timeout of 30 min (initial value). *Hypothesis:* yields richer command and payload capture than a Scanner policy.
- **Advanced:** maximum engagement, extended fake filesystem content and service responses, containment controls active. *Hypothesis:* prolongs engagement and increases IOCs per session.

Escalation and de-escalation. Classification is repeated as a session unfolds (e.g., after every k commands, with k a tunable parameter). A session may move up one level when new evidence raises the predicted class, and moves down only after a configurable number of consecutive lower predictions, to avoid oscillation.

Kill Switch. The Kill Switch is a containment mechanism, not an engagement mechanism. It terminates a session and blocks its source when predefined safety conditions hold, such as attempted outbound connections, resource exhaustion by an automated client, or exceeding a hard session limit. The idea of separating bot-like from human-like behavior is motivated by the bot-versus-human classifier in [6]; the specific triggers here are our design.

TABLE IV
XGBOOST HYPERPARAMETERS (DESIGN CHOICES)

Parameter	Initial	Grid search range
`max_depth`	6	{4, 6, 8}
`learning_rate`	0.1	{0.05, 0.1, 0.2}
`n_estimators`	100	{100, 200, 300}
Early stopping rounds	10	fixed
Objective	softprob	softprob vs. focal loss
Focal γ	—	[TO BE FILLED: range]

F. Intelligence Layer: Dashboard

The dashboard presents a live attack feed, threat-level distribution, top source IP addresses, per-session timelines, and extracted IOCs. Real-time alerting with a Flask dashboard follows [5]. Reference [6] reports visualization latency under 120 ms; we adopt this as a comparison point and will measure our own latency rather than assume it (Section V-A).

IV. IMPLEMENTATION

A. Technology Stack

The prototype uses Python 3.10 or later; Cowrie in Docker; XGBoost, scikit-learn, pandas, and NumPy for modeling; FastAPI for a classification endpoint; Flask or Streamlit for the dashboard; and SQLite (PostgreSQL for larger deployments) for storage. [VERIFY: pin exact library versions used in the final experiments]

B. Dataset

Training and evaluation will use the Shell Honeypot Attack Request–Response dataset [10] (12,923 commands from 2021–22 and 4,236 from 2024). Because that dataset consists of command and response records, timing and authentication features in Table I may be unavailable for it. Two options are planned: restrict the feature set to those derivable from the data, or collect sessions from our own Cowrie deployment over a period of several weeks. MURHCAD [11] (132,425 events, 2,438 unique IPs, 95 countries) is a candidate for supplementary Cowrie sessions. [TO BE FILLED: final dataset used, session counts per class after labeling]

C. Feature Extraction Pipeline

Listing 1 shows the core of session-level extraction from Cowrie JSON events. Pattern lists for privilege escalation and lateral movement are illustrative and will be published with the repository.

D. Model Training

Table IV lists the hyperparameters. These are initial *choices*, not tuned results. Data is split 80/20 with stratification by class; model selection uses 5-fold stratified cross-validation on the training portion, with early stopping on a validation fold. Grid search over the ranges in Table IV precedes the final fit, and the test split is used once.

Listing 1. Session feature extraction (excerpt).

```
import json, math, re
from collections import Counter, defaultdict
from datetime import datetime

PRIV = re.compile(r"\b(sudo|su\s|chmod\s+\+s)")

def entropy(s):
    c = Counter(s)
    n = len(s) or 1
    return -sum(v/n * math.log2(v/n) for v in c.values())

def load_sessions(path):
    sessions = defaultdict(list)
    with open(path) as f:
        for line in f:
            e = json.loads(line)
            sessions[e["session"]].append(e)
    return sessions

def features(events):
    parse_ts = lambda e: datetime.fromisoformat(
        e["timestamp"].replace("Z", "+00:00"))
    cmds = [e for e in events
            if e["eventid"] == "cowrie.command.input"]
    text = [e["input"] for e in cmds]
    ts = [parse_ts(e) for e in cmds]
    gaps = [(b - a).total_seconds() for a, b in zip(ts, ts[1:])]
    return {
        "command_count": len(text),
        "unique_commands": len(set(text)),
        "command_diversity": len(set(text)) / max(len(text), 1),
        "inter_command_delay_avg": sum(gaps) / len(gaps) if gaps
      else -1,
        "failed_auth_attempts": sum(
            e["eventid"] == "cowrie.login.failed" for e in
     events),
        "privilege_escalation": sum(bool(PRIV.search(x)) for x
     in text),
        "entropy_score": sum(map(entropy, text)) / max(len(text)
     , 1),
    }
```

E. Adaptive Response Implementation

The response engine is a rule table keyed by predicted class, read by a small service that adjusts Cowrie parameters (session timeout, filesystem profile) and invokes container-level controls for the Kill Switch. [VERIFY: confirm which Cowrie configuration options can be changed per session at runtime versus requiring a restart; if per-session changes are not supported, implement the policy through a proxy or a custom Cowrie output/command plugin].

F. Dashboard Implementation

A Flask service reads from the session database and serves a JSON API consumed by the page, which polls or subscribes for updates. Alerts are generated when a session reaches the Advanced class or triggers the Kill Switch.

G. Reproducibility

Code, pattern lists, labeling script, and configuration will be released in a public repository. [TO BE FILLED: repository URL]

V. EVALUATION PROTOCOL

This section specifies what will be measured. No results are reported in this draft.

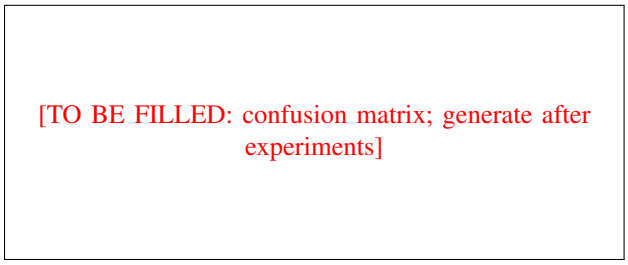


Fig. 3. Confusion matrix of the XGBoost classifier on the test split.

A. Experimental Setup

Experiments will use the dataset(s) of Section IV-B with the stratified 80/20 split and 5-fold cross-validation of Section IV-D. [TO BE FILLED: hardware] and [TO BE FILLED: exact software versions]. Engagement experiments require live or replayed attacker sessions against two deployments: a static Cowrie configuration and ML_Honey_trap. Classification latency and dashboard latency are measured end to end.

B. Hypotheses

- **H1:** Adaptive deception increases mean engagement duration compared with a static Cowrie configuration.
- **H2:** XGBoost achieves accuracy above 95% on the derived three-class problem. This is a target, and its success depends in part on the labeling heuristic (Section V-G).
- **H3:** Threat-level-specific responses improve threat-intelligence quality, measured as IOCs per session and MITRE ATT&CK technique coverage.

C. Metrics

Classification: accuracy, per-class precision, recall, and F1, and weighted and macro averages; confusion matrix. *Engagement:* mean and median session duration and commands per session. *Intelligence quality:* count of unique IOCs (IPs, URLs, hashes) per session and the number of distinct MITRE ATT&CK techniques [15] observed. *Efficiency:* inference latency and dashboard update latency.

D. Baselines

(1) Static Cowrie with a fixed policy; (2) a rule-based classifier using thresholds on command count and signature matches; (3) Random Forest on the same features, motivated by [4], [5].

E. Comparison Table

Table V lists reported figures from prior work next to ML_Honey_trap placeholders. Prior figures are quoted as reported by their sources and *are not directly comparable*: tasks, datasets, and metrics differ (e.g., [4] uses NSL-KDD, and [3] reports detection rather than three-class accuracy).

TABLE V
COMPARISON WITH REPORTED FIGURES

System	Reported metric	Source
Static Cowrie	[TO BE FILLED: our measurement]	ours
Rule-based	[TO BE FILLED: our measurement]	ours
Random Forest	[TO BE FILLED: our measurement]; [4]: 17,354 TP, 235 FN, 196 FP (NSL-KDD)	ours; [4]
RL honeypot	97% detection accuracy, 3.8% FP	[3]
RL+LLM	+23% engagement reward; +55 pp intent accuracy vs. Wazuh	[9]
HoneyTrap	100% credential capture; $<$120 ms visualization	[6]
GMNH	Not reported in [2]	[2]
Asgard2.0	Not reported in [7]	[7]
Proposed	[TO BE FILLED: accuracy, F1, engagement, IOCs/session]	ours



Fig. 4. Comparison of ML_Honey_trap with baselines on accuracy, F1, and engagement duration.

F. Ablation Study

The following variants will be run under the same split: (a) without the adaptation layer (classification only); (b) without the payload-level feature group; (c) leave-one-feature-group-out over session-level features; (d) Random Forest in place of XGBoost; (e) focal loss versus weighted cross-entropy; (f) alternative V_i thresholds for the label rule. [TO BE FILLED: results for each variant]

G. Threats to Validity

Label circularity. Labels derive from V_i, and features such as signature matches may correlate with V_i by construction; high accuracy could partly reflect the labeling rule rather than real attacker skill. We will report results with signature-based features removed. *Dataset age and drift.* The commands in [10] span 2021–22 and 2024, and attacker behavior changes over time; we will test training on one period and evaluating on the other. *Lab versus production.* Controlled evaluations can overstate production performance [1]; live deployment is future work. *Dataset provenance.* Results on one dataset may not transfer to others such as [11].

VI. DISCUSSION

A. Design Implications

If the hypotheses hold, a continuously updated threat estimate lets defenders allocate engagement where it yields intelligence and contain risk where it does not. For security

operations teams, a three-level output is simple to act on. For researchers, an explicit labeling rule and published code make comparison and criticism possible. Whether three classes are sufficient is itself a question the ablations and threshold-sensitivity analysis will inform.

B. Limitations

The framework inherits the lab-versus-production gap noted in [1]. The labeling heuristic is not validated against expert annotation, and session-level aggregation of a command-level severity index is a simplification. Pattern lists for privilege escalation and lateral movement are incomplete by nature. Scalability to many concurrent sessions has not been assessed.

C. Attacker Adaptation

Attackers may use automated tools, including AI-based techniques, to recognize honeypots [8]. A response engine that behaves deterministically per class could itself become a fingerprint. Randomizing parts of the policy and incorporating learned response generation as in [8] are directions to explore.

D. Ethical Considerations

Operating a honeypot raises legal and privacy questions that vary by jurisdiction, including the handling of source IP addresses and attacker-supplied data. The Kill Switch and outbound filtering exist to prevent the system from being used to harm third parties. Collected data should be stored securely and shared only in a minimized form, and any disclosure of discovered malware or infrastructure should follow responsible-disclosure practice.

VII. Conclusion and Future Work

A. Summary

This draft presented ML_Honey_trap, a four-layer framework that links session-level threat classification with adaptive engagement in a Cowrie-based honeypot. We specified a 15-feature representation, an XGBoost classifier with optional focal loss, an explicit heuristic that derives Scanner, Amateur, and Advanced labels from the severity index of an existing dataset, and a response engine with a containment Kill Switch. We also defined an evaluation protocol with three hypotheses, baselines, and ablations. The experiments remain to be run, and the paper's claims will be set by their outcome.

B. Future Work

Planned directions are: reinforcement-learning-based response selection in the spirit of [7]; LLM-generated deceptive responses as explored in [9]; federated learning across distributed honeypots; production deployment with long-term monitoring; and support for additional protocols beyond SSH and Telnet, such as HTTP, FTP, and SQL [6].

Acknowledgment

The authors thank PRAM Educate IT Software LLP for their support.

References

[1] "AI-Assisted Honeypot for Threat Adaptation and Mitigation," IEEE, Mar. 2026. [VERIFY: authors, venue, DOI]
[2] "GhostMesh Neuro-Honeynet (GMNH)," IEEE, Feb. 2026. [VERIFY: authors, venue, DOI]
[3] "A Dynamic AI-Powered Honeypot Framework," IEEE, Oct. 2025. [VERIFY: authors, venue, DOI]
[4] "Adaptive Honeypot Simulation & Intrusion Detection Using Supervised ML," IEEE, Nov. 2025. [VERIFY: authors, venue, DOI]
[5] "Forensic Analysis of AI-Driven Cyber Deception Networks," Zenodo, May 2026. [VERIFY: authors, DOI]
[6] "HoneyTrap: AI-Powered Honeypot for Cyberattack Detection," IJERT, 2026. [VERIFY: authors, volume, pages]
[7] "Asgard2.0: Adaptive Self-Guarded Risk-Aware Honeypot Using DRL," ACM ESORICS 2025. [VERIFY: authors, pages, DOI]
[8] "Q-Cowrie: An Adaptive Honeypot," ACM IJIS, Feb. 2026. [VERIFY: authors, volume, pages, DOI]
[9] "Reinforcement Learning Meets LLM Honeypots," IEEE, Sep. 2026. [VERIFY: authors, venue, DOI]
[10] "Shell Honeypot Attack Request–Response Dataset," HuggingFace. [VERIFY: authors, year, DOI]
[11] "MURHCAD: Multi-Regional Cloud Honeypot Dataset," arXiv, 2026. [VERIFY: authors, DOI]
[12] "SoK: Honeypots & LLMs," IEEE, 2026. [VERIFY: authors, venue, DOI]
[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794.
[14] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl.*, 2009.
[15] B. Strom *et al.*, "MITRE ATT&CK: Design and philosophy," MITRE Tech. Rep., 2018.
[16] M. Oosterhof, "Cowrie SSH/Telnet honeypot," GitHub repository. [VERIFY: URL, access date]